

An Allocator-Aware `inplace_vector`

Document #: P3160R1
Date: 2024-03-08 20:42 EST
Project: Programming Language C++
Audience: LEWG
Reply-to: Pablo Halpern
<phalpern@halpernwrightsoftware.com>

Contents

- 1 Abstract
- 2 Change Log
- 3 Motivation
 - 3.1 General Motivation for Allocator-Aware Types
 - 3.2 Motivation for an Allocator-Aware `inplace_vector`
- 4 Proposal Summary
 - 4.1 Design Decision for Discussion
- 5 Alternatives Considered
 - 5.1 Option 1: `inplace_vector<class T, size_t N, class Alloc = std::allocator<T>>`
 - 5.2 Option 2: `inplace_vector<class T, size_t N>`
 - 5.3 Option 3: `inplace_vector<class T, size_t N, class Alloc = see below>`
 - 5.4 Option 4: `basic_inplace_vector<class T, size_t N, class Alloc>`
- 6 Performance and Compile-Time Costs
 - 6.1 About the Performance Tests
 - 6.2 Methodology
 - 6.3 Runtime Results
 - 6.4 Compile-Time Results
- 7 Wording
- 8 References

§ 1 Abstract

The `inplace_vector` proposal, [P0843R10], is moving forward without allocator support. This paper proposes that `inplace_vector` *should* have allocator support and explores the pros and cons of adding such support directly into `inplace_vector` vs. into a separate `basic_inplace_vector` class template. This proposal is distinct from [P0843R10] so that the latter can move forward quickly while allocator-specific policies are still being resolved.

§ 2 Change Log

R1

- [Proposal Summary](#) section added
- *Design Options* section renamed to [Alternatives Considered](#)
- *Compile-Time Data* section significantly expanded and renamed to [Performance and Compile-Time Costs](#)
- Miscellaneous copy editing

R0

- Initial version

§ 3 Motivation

§ 3.1 General Motivation for Allocator-Aware Types

Note: The text below is borrowed nearly verbatim from [P3002R1], which proposes a general policy for when types should use allocators.

Memory management is a major part of building software. Numerous facilities in the C++ Standard library exist to give the programmer maximum control over how their program uses memory.

- `std::unique_ptr` and `std::shared_ptr` are parameterized with *deleter* objects that control, among other things, how memory resources are reclaimed.
- `std::vector` is preferred over other containers in many cases because its use of contiguous memory provides optimal cache locality and minimizes allocate and deallocate operations. Indeed, the LEWG has spent a lot of time on `flat_set` ([P1222R0]) and `flat_map` ([P0429R3]), whose underlying structure defaults, for this reason, to `vector`.
- Operators `new` and `delete` are replaceable, giving programmers global control over how memory is allocated.

- The C++ object model makes a clear distinction between an object’s memory footprint and its lifetime.
- Language constructs such as `void*` and `reinterpret_cast` provide fine-grained access to objects’ underlying memory.
- Standard containers and strings are parameterized with allocators, providing object-level control over memory allocation and element construction.

This fine-grained control over memory that C++ gives the programmer is a large part of why C++ is applicable to so many domains — from embedded systems with limited memory budgets to games, high-frequency trading, and scientific simulations that require cache locality, thread affinity, and other memory-related performance optimizations.

An in-depth description of the value proposition for allocator-aware software can be found in [P2035R0]. Standard containers are the most ubiquitous examples of *allocator-aware* types. Their `allocator_type` and `get_allocator` members and allocator-parameterized constructors allow them to be used like building blocks that can be combined and nested as necessary while retaining full programmer control over how the whole assembly allocates memory. For *scoped allocators* (i.e., those that apply not only to the top-level container, but also to its elements), having each element of a container support a predictable allocator-aware interface is crucial to giving the programmer the ability to allocate all memory from a single memory resource, such as an arena or pool. Note that the allocator is a *configuration* parameter of an object and does not contribute to its value.

In short, four principles underlie this policy proposal.

1. **The Standard Library should be general and flexible.** The user of a library class should have control, to the greatest extent possible, over how memory is allocated.
2. **The Standard Library should be consistent.** The use of allocators should be consistent with the existing allocator-aware classes and class templates, especially those in the containers library.
3. **The parts of the Standard Library should work together.** If one part of the library gives the user control over memory allocation but another part does not, then the second part undermines the utility of the first.
4. **The Standard Library should encapsulate complexity.** The generic application of allocators with maximum flexibility is potentially complex and is best left to the experts implementing the Standard Library. Users can choose their own subset of desirable allocator behavior only if the underlying Library classes allow them to choose their preferred approach, whether it be stateless allocators, statically typed allocators, polymorphic allocators, or no allocators.

§ 3.2 Motivation for an Allocator-Aware `inplace_vector`

The main motivations for `inplace_vector` as a whole are to give the user control over memory allocation. If such a type did not fit into an existing infrastructure that is also designed to give the user control over memory use, that would certainly seem ironic.

Although the objects stored in an `std::inplace_vector`, as proposed in [P0843R10], can be emplaced with any set of valid constructor arguments, including allocator arguments, the fact that the `inplace_vector` itself is not allocator aware prevents it from working consistently with other parts of the Standard Library, specifically those parts that depend on *uses-allocator construction* (section [allocator.uses.construction]) in the Standard:

```
pmr::monotonic_buffer_resource rsrc;
pmr::polymorphic_allocator<> alloc{ &rsrc };
using V = inplace_vector<pmr::string, 10>;
V v = make_obj_using_allocator<V>(alloc, { "hello", "goodbye" });
assert(v[0].get_allocator() == alloc); // FAILS
```

Even though an allocator is supplied, it is not used to construct the `pmr::string` objects within the resulting `inplace_vector` object because `inplace_vector` does not have the necessary hooks for `make_obj_using_allocator` to recognize it as being allocator aware. Note that, although this example and the ones that follow use `pmr::polymorphic_allocator`, the same issues would apply to any scoped allocator.

Uses-allocator construction is rarely used directly in user code. Instead, it is used within the implementation of standard containers and scoped allocators to ensure that the allocator used to construct the container is also used to construct its elements. Continuing the example above, consider what happens if an `inplace_vector` is stored in a `pmr::vector` compared to truly allocator-aware type (`pmr::string`) in a `pmr::vector`:

```
pmr::vector<pmr::string> vs(alloc);
pmr::vector<V>          vo(alloc);

vs.emplace_back("hello");
vo.emplace_back({ "hello" });

assert(vs.back().get_allocator() == alloc); // OK
assert(vo.back()[0]->get_allocator() == alloc); // FAILS
```

An important invariant when using a scoped allocator such as `pmr::polymorphic_allocator` is that the same allocator is used throughout an object hierarchy. `inplace_vector` cannot ensure that this invariant is preserved, even if each element is originally inserted with the correct allocator, because `inplace_vector` does not remember the allocator used to construct it and thus cannot supply the allocator to new elements. The abstraction has become unreliable and the programmer must manually enforce the scoped-allocator invariant.

§ 4 Proposal Summary

The proposal offered here is to make `inplace_vector` an *allocator-aware container* as described in 24.2.2.5 [\[container.alloc.reqmts\]](#)¹ and as specified below.

- The class template would have a third template argument, `Allocator`, which would default to `std::allocator<T>`. The `std::pmr` namespace would also contain an alias:

```
template <class T, size_t N, class Allocator = allocator<T>>
class inplace_vector; // partially freestanding

namespace pmr
{
    template <class T, size_t N>
    using inplace_vector = std::inplace_vector<T, N,
        polymorphic_allocator<T>>;
}
```

- The interface would add `allocator_type` and `get_allocator` members:

```
using allocator_type = Allocator;

constexpr allocator_type get_allocator() const;
```

- Every constructor would have either an overload or a default argument allowing the allocator to be passed as its last argument:

```
constexpr inplace_vector() noexcept;
constexpr explicit inplace_vector(const allocator_type& a)
    noexcept;
constexpr explicit inplace_vector(size_type n); // freestanding-
    deleted
constexpr inplace_vector(size_type n,
    const allocator_type& a); //
    freestanding-deleted
constexpr inplace_vector(size_type n, const T& value,
    const allocator_type& a = {}); //
    freestanding-deleted
// etc..
```

- If the element type, `T`, uses the specified allocator (i.e., `uses_allocator_v<T, Allocator>` is true), then the allocator is passed to the constructor of `T` via *uses-allocator construction* (20.2.8.2 [\[allocator.uses.construction\]](#)) when elements are inserted into the `inplace_vector`.

§ 4.1 Design Decision for Discussion

For wrapper types such as `tuple<T>`, an allocator passed to the constructor is passed through to the wrapped `T` object via *uses-allocator construction* (20.2.8.2 [\[allocator.uses.construction\]](#)), regardless of whether the allocator is a scoped allocator. The reasoning is that, since the tuple does not itself allocate memory, passing in an allocator that is compatible with `T` but which is not passed to the wrapped `T` object makes no sense. The same logic applies to the proposed `basic_optional` and `basic_variant` templates proposed in [\[P2047R7\]](#) and [\[P3153R0\]](#), respectively.

On the other hand, the requirements on an allocator-aware container in 24.2.2.5 [\[container.alloc.reqmts\]](#) indicate that elements should always be constructed using `allocator_traits<Allocator>::construct`. A nonintuitive downside of following this convention is that only an allocator having a special `construct` method, such as a scoped allocator, would be used to construct elements; other allocator types would effectively be ignored (but they might take up space in the object footprint). The main upside, however, is that the existing wording in 20.2.8.2 [\[allocator.uses.construction\]](#) would apply unchanged, including the definitions of *Cpp17DefaultInsertable*, *Cpp17MoveInsertable*, *Cpp17CopyInsertable*, *Cpp17EmplaceConstructible*, and *Cpp17Erasable*.

To summarize:

- *Uses-allocator construction* is consistent with other nonallocating types that contain allocating types.
- `allocator_traits<Allocator>::construct` is consistent with other containers, including `std::vector`.
- For most scoped allocators, including `pmr::polymorphic_allocator`, the two designs are equivalent. If a scoped allocator uses different allocation mechanisms at different nesting levels, however, `construct` will use the top level mechanism to construct objects, but not to allocate memory.
- Workarounds employing the `scoped_allocator_adaptor` would let each design behave like the other.

The design in this document constructs elements via *uses-allocator construction*, but LEWG should consider this issue carefully, including in the context of a larger policy discussion.

§ 5 Alternatives Considered

Several possible designs for an allocator-aware `inplace_vector` have been considered. Option 1 — with none of its suboptions — is being proposed here.

§ 5.1 Option 1: `inplace_vector<class T, size_t N, class Alloc = std::allocator<T>>`

The allocator would be stored in the object and returned by `get_allocator()`. If `uses_allocator_v<T, Alloc>` is true, then elements are constructed via *uses-allocator construction* with the supplied allocator. For `std::allocator`, no space would be need to be taken up; in fact, consuming no space should be a *requirement* so that `sizeof(inplace_vector<T, N>)` is guaranteed to be $N * \text{sizeof}(T)$, i.e., no size overhead for the default allocator case.

Pros: This option offers the simplest design, is most consistent with other containers, and has zero runtime overhead for the default case.

Cons: If `uses_allocator_v<T, Alloc>` is false and `Alloc` is a nonempty class, then space is wasted storing an unused allocator.

Suboptions for reducing wasted footprint storage for non-AA elements:

- a. In this case, `inplace_vector` could be non-allocator-aware, `allocator_type` and `get_allocator()` would not be defined, and no constructors would accept an allocator argument.
- b. Supplying an allocator other than `std::allocator` for a type that cannot use it could be ill-formed.

Unfortunately, both sub-options would make generic programming somewhat more difficult.

§ 5.2 Option 2: `inplace_vector<class T, size_t N>`

If `T::allocator_type` exists, then `inplace_vector<class T, size_t N>::allocator_type` would also exist, as would `get_allocator()` and allocator-accepting constructors. Otherwise, the instantiation would not be allocator aware.

Pros: The user cannot accidentally forget to specify an allocator parameter when using allocator-aware element types.

Cons: If `T` is allocator aware but the user doesn't want to take advantage of that quality, an allocator is stored unnecessarily. Worse, no obvious workarounds are available to avoid storing and using an allocator. This approach is inconsistent with other containers; nowhere else in the Standard does a container-element's allocator type influence the allocator type of the container itself.

§ 5.3 Option 3: `inplace_vector<class T, size_t N, class Alloc = see below >`

The *see below* type is `T::allocator_type` if such a type exists and `std::allocator<byte>` otherwise.

Pros: Accidentally forgetting to specify an allocator parameter when using allocator-aware types is easily avoided (as in Option 2), but automatic allocator selection can be easily overridden.

Cons: Automatically detecting the element's allocator type is novel and inconsistent with other containers. When an allocator is not supplied by the user, the resulting `inplace_vector` might be unintentionally allocator aware, incurring performance penalties. Such accidents are the flip side of the Pros for this option; some accidents are avoided while different accidents can now occur.

§ 5.4 Option 4: `basic_inplace_vector<class T, size_t N, class Alloc>`

This approach has a separate template, `basic_inplace_vector`, that is the same as the allocator-aware `inplace_vector` proposed in Options 1 or 3, but without affecting the interface of `inplace_vector` proposed in [P0843R10].

Pros: This option reduces complications in the `inplace_vector` specification and implementation and potentially reduces compile time compared to some of the preceding options.

Cons: `inplace_vector` and `basic_inplace_vector` are separate, incompatible, types. The user needs to make a decision, especially in generic code, regarding whether they ever expect to have allocator-aware elements. Choosing the shorter name is tempting, but comes at the expense of breaking scoped allocation in generic code. This option increases *overall* specification complexity and implementation because now *two* classes must be specified, implemented, and tested, so `basic_inplace_vector` is just as complicated as Options 1 or 3, above.

§ 6 Performance and Compile-Time Costs

§ 6.1 About the Performance Tests

When people talk about “low- or no-cost abstractions,” they usually mean runtime costs, but we are increasingly seeing concern about compile-time costs, as metaprogramming and concepts allow us to express ever-more complex constructs. Collecting data on both the runtime and compile-time costs of adding generic allocator awareness to `inplace_vector` is, therefore, important to aid the Committee in evaluating the options.

I performed a set of experiments with a subset of an `inplace_vector` implementation, looking at the generated assembly code and measuring compile times for a non-allocator-aware version, as proposed in [P0843R10], compared to Options 1, 2, and 3 described above. Option 4 was not measured since it involves a completely separate class template, `basic_inplace_vector`, that presumably would not affect the performance of the unmodified `inplace_vector` except for the nominal cost of reading the text of the header file at compilation time.

The [runtime](#) and [compile-time results](#) are shown below, after the [Methodology](#) section.

§ 6.2 Methodology

Allocator awareness was added to a partial implementation of [P0843R10]. To save effort, only functionality needed for the test program was implemented. For the allocator-aware versions of `inplace_vector`, compile time was reduced marginally by implementing a partial specialization for the common case of `std::allocator`. The test driver was

designed to maximize the number of distinct instantiations of `inplace_vector`, its constructors, and its insertion methods (especially `push_back`), thus producing a near-worst-case scenario for any template-instantiation penalty caused by the inclusion of the allocator machinery.

Compilation was launched from a Python script that compiled but did not link five identical files, each of which `#included` the test program. Compilation was timed for each combination of implementation option, compiler, and element types (allocator-aware, non-allocator-aware or blended/both). The script also generated one assembly listing and one executable for each configuration. The generation of the assembly and executable files was not included in the timing tests. The executable was run as a *smoke test* to ensure that the code ran and passed some simple tests, but the run time was too short for any meaningful comparisons and so was not measured; runtime performance equivalence was deduced based on assembly-language equivalence. Timing jitter was on the order of 3-4%, so timing differences under 5% should be ignored.

The tests were run on a Lenovo ThinkPad running Ubuntu 22.04 Linux within the Windows Subsystem for Linux. The CPU was an 11th Gen Intel(R) Core(TM) i7-11850H running at 2.50GH with 64GB RAM and a 512GB SSD. The compilers used in the test were GCC 11.4.0 using `-std=C++23` and Clang 14.0.0 using `-std=C++2b`. All tests were conducted with level `-O2` optimization. The experimental sources and scripts can be found at https://github.com/phalpern/WG21-halpern/tree/main/P3160-AA-inplace_vector/.

§ 6.3 Runtime Results

The test program was not designed to measure run time. Although a smoke test was performed on each option to test that it produced correct results, run times were too short to produce meaningful comparisons. Instead, I compared assembly-language output from each configuration to see if the presence of the allocator machinery produced any difference in the generated output for both allocator-aware element types and non-allocator-aware element types. Comparing GCC-to-GCC and Clang-to-Clang produced the same results, shown in the table below.

Comparing assembly-language output against P0843R10

	Non-allocator-aware T	Allocator-aware T
Option 1	Identical	Identical
Option 2	Identical	<i>Different</i>
Option 3	Identical	<i>Different</i>

From this table, we can infer that the allocator machinery incurs no runtime penalty when the element type does not use an allocator. Moreover, Option 1 produced no penalty when the element types *do* use an allocator, provided that the allocator was not specifically provided to `inplace_vector`, i.e., when `inplace_vector` was instantiated with the default allocator (`std::allocator<T>`). Options 2 and 3 produced different results for allocator-

aware element types, as expected, since the default allocator type for `inplace_vector<T, N>` is `T::allocator_type` in both cases, rather than `std::allocator<T>`.

From these results, we can conclude that Option 1 produces no runtime penalty for using the default allocator parameter; any penalty would be borne exclusively by users that deliberately specify an allocator argument.

§ 6.4 Compile-Time Results

The raw data shown in the tables below present the increase in user-mode compile time for each option when compared to the [P0843R10] status quo. They show that there is no appreciable compile-time penalty using any of the options when the element type is not allocator aware (e.g., `inplace_vector<int, N>`). Option 1 also had no penalty for allocator-aware element types when the allocator type was left at the default `std::allocator<T>`. Options 2 and 3 did exhibit a noticeable compile-time impact for allocator-aware elements (e.g., `inplace_vector<pmr::string, N>`). The additional compile time can be attributed to the machinery for passing the allocator to the elements when an allocator is provided as well as the metaprogramming needed to determine if `T::allocator_type` exists.

From these results, we can conclude that the allocator-aware `inplace_vector` described in Option 1 imposes no significant compile-time cost compared to a non-allocator-aware `inplace_vector`.

Raw compilation timing data

Compilation times for P0843R10

Compiler	Element Types (T)	Compilation Time (s)
g++	Non-Allocator-Aware	3.37
g++	Allocator-Aware	3.47
g++	Some AA, Some non-AA	5.14
clang++	Non-Allocator-Aware	3.87
clang++	Allocator-Aware	4.32
clang++	Some AA, Some non-AA	6.09

Compilation times for Option 1

Compiler	Element Types (T)	Compilation Time (s)	Increase
g++	Non-Allocator-Aware	3.4	1%
g++	Allocator-Aware	3.55	2%
g++	Some AA, Some non-AA	5.21	1%
clang++	Non-Allocator-Aware	3.89	1%
clang++	Allocator-Aware	4.33	0%
clang++	Some AA, Some non-AA	6.35	4%

Compilation times for Option 2

Compiler	Element Types (T)	Compilation Time (s)	Increase
g++	Non-Allocator-Aware	3.34	-1%
g++	Allocator-Aware	4.26	23%
g++	Some AA, Some non-AA	5.97	16%
clang++	Non-Allocator-Aware	3.96	2%
clang++	Allocator-Aware	5.34	24%
clang++	Some AA, Some non-AA	7.43	22%

Compilation times for Option 3

Compiler	Element Types (T)	Compilation Time (s)	Increase
g++	Non-Allocator-Aware	3.44	2%
g++	Allocator-Aware	4.29	24%
g++	Some AA, Some non-AA	5.89	15%
clang++	Non-Allocator-Aware	3.92	1%
clang++	Allocator-Aware	5.35	24%
clang++	Some AA, Some non-AA	7.34	21%

§ 7 Wording

Wording will be provided after initial LEWG discussion.

Design Decisions

- Are we are interested in an allocator-aware `inplace_vector`?
- Which interface (Options 1 – 4) do we want?
- How should the allocator be passed to the inserted elements: via *uses-allocator construction* or via `allocator_traits::construct`?

Procedural Decisions

How should I specify wording? I can think of three possible ways (listed from least to most effort).

1. Work with the authors of P0843 to integrate allocator wording.
2. Create a separate diff against P0843 wording.
3. Wait until P0843 has been merged with the WP and create a diff against the WP.

§ 8 References

- [P0429R3] Zach Laine. 2016-08-31. A Standard flat_map.
<https://wg21.link/p0429r3>
- [P0843R10] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. 2024-02-12. inplace_vector.
<https://wg21.link/p0843r10>
- [P1222R0] Zach Laine. 2018-10-02. A Standard flat_set.
<https://wg21.link/p1222r0>
- [P2035R0] Pablo Halpern, John Lakos. 2020-01-13. Value Proposition: Allocator-Aware (AA) Software.
<https://wg21.link/p2035r0>
- [P2047R7] Nina Ranns, Pablo Halpern Ville Voutilainen. 2024-02-15. An allocator-aware optional type.
<https://wg21.link/p2047r7>
- [P3002R1] Pablo Halpern. 2024-02-15. Policies for Using Allocators in New Library Classes.
<https://wg21.link/p3002r1>
- [P3153R0] Nina Ranns, Pablo Halpern, Ville Voutilainen. 2024-02-15. An allocator-aware variant type.
<https://wg21.link/p3153r0>
-

1. All citations to the Standard are to working draft N4971 unless otherwise specified. ↩